

AmbiComp: A platform for distributed execution of Java programs on embedded systems by offering a single system image.

Johannes Eickhold · Thomas Fuhrmann · Bjoern Saballus ·
Sven Schlender · Thomas Suchy

Received: date / Accepted: date

Abstract Ambient Intelligence pursues the vision that small networked computers will jointly perform tasks that create the illusion of an intelligent environment. One of the most pressing challenges in this context is the question how one could easily develop software for such highly complex, but resource-scarce systems.

In this paper we present a snapshot of our ongoing work towards facilitating software development for Ambient Intelligence systems. In particular, we present the AmbiComp [1] platform. It consists of small, modular hardware, a flexible firmware including a Java Virtual Machine, and an Eclipse-based integrated development environment.

Authors occur in alphabetical order and not by contribution to this paper.

Johannes Eickhold, Bjoern Saballus
University of Karlsruhe
Department of Computer Science
System Architecture Group
Am Fasanengarten 5
76131 Karlsruhe
E-mail: eickhold,saballus@ira.uka.de

Thomas Fuhrmann
Technical University Munich
Boltzmannstrasse 3
85748 Garching
E-mail: fuhrmann@net.in.tum.de

Sven Schlender
Beecon GmbH
Albert-Nestler-Strasse 10
76131 Karlsruhe
E-mail: schlender@beecon.de

Thomas Suchy
Hochschule der Medien Stuttgart
Nobelstrasse 10
70569 Stuttgart
E-mail: suchy@hdm-stuttgart.de

Keywords Ambient Intelligence · Distributed Systems · Java Virtual Machine · Embedded Systems

1 Introduction

The AmbiComp project is concerned with the software development process for a network of smart products. Together, such smart products form a potentially large distributed system that consists of small embedded controllers. AmbiComp aims at providing an easy means for software engineers to develop and deploy Ambient Intelligence software in such networks. To this end, the AmbiComp partners have developed the concept of a so-called *Ambient Intelligence Control Unit* (AICU), which can serve as basic building block for smart products.

AICUs consist of one or more so-called sandwich modules (SMs). They equip an AICU with compute power, communication interfaces, and input/output lines for analog and digital signals. At least one of the SMs in an AICU executes a Java Virtual Machine. These AmbiComp Virtual Machines (ACVMs) are able to create a single system illusion across several physically separated AICUs.

AICUs can be programmed in Java. Application programming interfaces (APIs) such as the Connected Limited Device Configuration (CLDC) of the JavaME standard provide easy access to all the IO and communication interfaces. With regards to this aspect AICUs are similar to the SunSPOTs from SUN Microsystems Inc. and the PerKIT from Sentilla Corporation. But unlike those, a Java program running on the AmbiComp platform is not tied to one AICU, only. AmbiComp applications may rather run on multiple AICUs so that a piece of code can transparently access phys-

ically separated sensors and actuators. Thus the software developer can concentrate on the application logic without the need to bother about programming a distributed system.

In this paper, we focus on those aspects that are relevant to the developer of smart products. In particular, we explain the hardware and software of AICUs, including the Eclipse-based integrated development environment (IDE). The details of the self-organizing distributed operation of the AICUs are beyond the scope of this paper.

This paper is structured as follows: Section 2 presents the modular AmbiComp hardware concept and the BIOS low-level hardware abstraction layer. Section 3 describes the AmbiComp Virtual Machine and the additionally required pre-processing tool. Section 4 gives an overview description of the Eclipse plug-in that supports the AmbiComp developer. Section 5 concludes this paper with a summary and an outlook to future work.

2 AmbiComp Hardware and BIOS Concepts

As described in section 1, the AmbiComp project aims at creating small, embedded devices (“AICUs”) that can be used to control smart products in an ambient intelligence environment. Such environments may have vastly differing properties, and the smart products may be used for a wide range of applications.

The AmbiComp project takes this into account with a modular approach: AICUs consist of heterogeneous sandwich modules (SMs) that can be stacked to jointly form an AICU. This modular design enables the developer of a smart product to create AICUs that match the respectively required purpose.

The SMs in an AICU are connected via pin-headers. Stacking the SMs thus implicitly creates a backplane. It provides the inter-SM communication channel and supplies each SM with power. SMs with a CPU are called *intelligent SMs*. An AICU must contain at least one intelligent SM.

A hardware abstraction layer (“BIOS”) abstracts from the different hardware features of the SMs. It offers a general interface to all kinds of hardware resources which may or may not be available on a given SM. This allows the ACVM to be independent of SMs.

2.1 Sandwich Module Overview

Sandwich modules offer a wide range of different hardware functionalities. They can be divided into four groups: computation, communication, input/output,

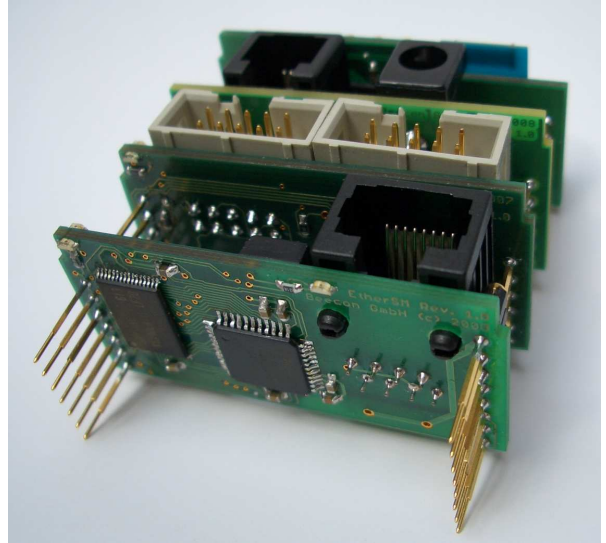


Fig. 1 Ambient Intelligent Control Unit

and power. Each SM combines different characteristics of the four functional groups. The developer of a smart product can add SMs to the AICU to increase the AICU’s functional range. She can also add SMs to increase the performance of an AICU. Thereby, an AICU can be tailored to a particular smart product where it then supports a potentially wide scale of applications.

Currently, the AmbiComp project has developed a basic set of four different SMs. They are all based on the Atmel AVR ATmega2561 RISC processor. SMs with a more powerful micro-controller are planned for the near future. Table 1 gives an overview of currently available SMs.

- The Input-Output Sandwich Module (IOSM) offers several general purpose input-output (IO) lines. It has a 16MHz controller to allow fast IO processing, but it has no external SRAM. Its two 10 pin connectors comply with the Roboternetz de-facto standard [2]. Each pin offers different functionalities. All pins can serve as digital IO or analog input. Four of the pins can alternatively act as analog output.
- The Bluetooth Sandwich Module (BTSM) hosts a Bluetooth 2.0 transceiver. The controller runs at 7.37MHz to match the serial interface of the transceiver. The BTSM has an additional 512KiB external SRAM, e.g. to buffer communication streams.
- The Ethernet Sandwich Module (EtherSM) comes with a 10MBit/s Ethernet transceiver, an 8MHz controller, and 512KiB external SRAM.
- The Power-over-Ethernet Sandwich Module (PoeSM) is the power supply for the AICU. When packed together with an EtherSM, it provides two

Table 1 Functional Groups of the AmbiComp Sandwich Modules

	CPU	Clock	ext. RAM	Flash	Communication	IO	Power	Power Supply
IOSM	8-bit RISC	16MHz	-	256KiB	-	16x dig. IO 16x ADC, 4x DAC	5V	-
BTSM	8-bit RISC	7.37MHz	512KiB	256KiB	Bluetooth 2.0	-	3.3V	-
EtherSM	8-bit RISC	8MHz	512KiB	256KiB	10MBit/s Ethernet	-	3.3V	-
PoeSM	-	-	-	-	-	-	-	3.3V/5V

voltage levels, 3.3V and 5V, to serve the different SM requirements.

As described above, all SMs in one AICU are connected via the backplane. This backplane offers an I^2C bus for the inter-SM communication between intelligent SMs. Additionally, the backplane provides the two typical voltage levels 3.3V and 5V to supply all SMs in the AICU. Note that currently only the PoeSM is able to supply the AICU with power, but further power supplies such as rechargeable batteries are planned for the near future.

2.2 The BIOS as Hardware Abstraction Layer

Each intelligent SM executes an instance of the AmbiComp Virtual Machine. It is independent of the underlying hardware because the BIOS provides generic interfaces to the hardware of the different SMs. As a consequence, each SM must have its specially tailored BIOS that maps the functionality of the respective SM to this generic interface.

The BIOS has to meet a number of requirements to fully allow this generic hardware abstraction. It

- initializes the hardware, e.g. set IO lines into a defined state,
- handles interrupts,
- resides in the boot loader section of the flash memory, so that it can update the remaining flash memory,
- contains the hardware dependent parts of drivers, and
- offers a special maintenance interface, e.g. to update the firmware.

The BIOS API consists of seven different groups:

1. *Streaming* provides separate stream functions for each communication channel. Currently there exist streams for Ethernet, HCI (Bluetooth), I^2C and the backplane communication between the SMs in an AICU.
2. *IO* contains all functions needed to set or get data from the different pins offered by the SM. This includes the access to digital IO as well as analog-

digital and digital-analog converter functions, if available.

3. *Events* signal the application specific conditions. For example the successful sending of an Ethernet frame or the edge change on an interrupt enabled IO pin.
4. *Memory* functions offer a generic way to obtain information about the available memory and its layout on the respective SM. This includes for example the availability of external SRAM and the information about its organization such as a split into separate pages.
5. *Timer* functions provide a general interface to a hardware timer.
6. The *Backdoor* offers a low level access to the SM, e.g. information about the BIOS version and the respective hardware type of the SM. The backdoor is accessible at all times, even when no ACVM is present or the ACVM has failed. This allows software updates of the ACVM.
7. *Versioning* is the BIOS mechanism to ensure that the ACVM has been compiled and linked with the BIOS version that is present on the SM. The ACVM only starts execution if ACVM and BIOS match.

3 The AmbiComp Virtual Machine

This section gives an overview of how Java programs are executed in the ACVM. It also explains what is needed before this execution can commence.

The usual workflow of a Java programmer is to write a Java program, compile the respective source files with a Java compiler, and execute the resulting bytecode (“class files”) on a Java Virtual Machine (JVM).

This approach has some drawbacks when applied to JVMs on embedded systems:

- Java class files have a comparatively big memory footprint. For example, they contain clear text names of classes, fields, and methods to allow late binding in the JVM.
- The JVM has a comparatively big memory footprint, too, because it needs to perform bytecode verification and late binding.

In the AmbiComp project we try to bypass both these drawbacks with the help of a transcoder.

3.1 The Transcoder

The transcoder performs a pre-processing step. It has to be executed before a Java program can run on the ACVM. Transcoding converts regular Java class files into an AmbiComp-specific binary representation, the so-called BLOB file. Only BLOBs can be executed by the AmbiComp Virtual Machine. This is similar to Sun's Squawk VM which is used for the Sun SPOTs [3].

The transcoder statically links all classes that are needed for an application. Thereby it can eliminate clear text references to class names, field names, and method names. Java class files contain a so-called constant pool with plaintext identifiers, e.g. *java.lang.Object*. It allows dynamic binding i.e. "class loading". The AmbiComp transcoder replaces all these names with numerical identifiers that are unique only within a BLOB. This reduces the memory footprint of the BLOB as compared to the original class files. It also allows a more compact ACVM as compared to a full-blown JVM.

Moreover, static linkage allows thorough dead code elimination, because an application typically uses only a small fraction of the libraries it includes. In some cases such as e.g. getter/setter or wrapper methods the transcoder can save code by inlining (parts of) the respective method. The advantage of a reduced code size pays off in many AmbiComp scenarios. Scenarios that require versatile libraries shall be supported by a future variant of the transcoder. It will be able to create BLOBs that can be linked to other BLOBs. But in any case, an AICU will not contain predeployed standard libraries. The developer can fully determine which BLOB or BLOBs shall be stored in an AICU.

Finally, the transcoder produces a more suitable binary representation of the Java bytecodes that reflects the limitations of the ACVM. It is particularly designed to less interpretation overhead. This leads to a significantly smaller memory footprint of the ACVM.

3.2 Properties of the ACVM

The AmbiComp Virtual Machine (ACVM) is specially designed for the resource-limited micro-controllers of the intelligent sandwich modules. The AmbiComp software stack (cf. fig. 2) is self-contained.

The ACVM does not require a real operating system. The AmbiComp OS-Layer is only a collection of platform-independent protocol implementations. The BIOS only abstracts from the different hardware of the SMs. Other operating system functions such as

scheduling or memory management are entirely up to the ACVM.

The ACVM can execute several single- or multi-threaded Java applications in parallel. The underlying thread-level concurrency is achieved by a priority round-robin scheduler. The ACVM memory manager guarantees a logical memory protection. The ACVM also provides a concurrent, real-time capable mark-and-sweep garbage collector.

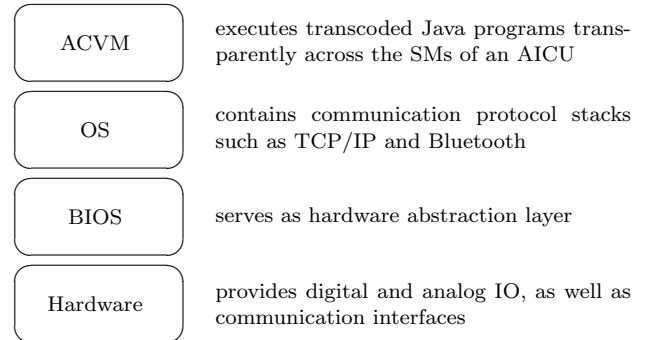


Fig. 2 The AmbiComp Hardware and Software Platform

3.3 Single System Illusion

An AmbiComp network connects heterogeneous AICUs. A *single system illusion (SSI)* hides the distributed and potentially heterogeneous nature of such a network. The SSI enables a programmer to write applications for the network as if it was a single system.

In networks where no such SSI is available, the programmer has to explicitly think about the following facts:

- Which resources are available in the network?
- On which nodes are these resources located?
- Which are the different components of the distributed application?
- How do these components communicate (RPC, RMI, proprietary protocol, ...)?
- How can components be discovered?

Resource requirements and the chosen communication mechanism induce mappings of components to network nodes. The mapping is used in the deployment phase to install the components onto nodes. The programmer has to craft the communication between components explicitly into the code.

An SSI shifts this communication from explicit (i.e. provided by the programmer) to implicit (i.e. provided as a feature of the execution environment). This allows

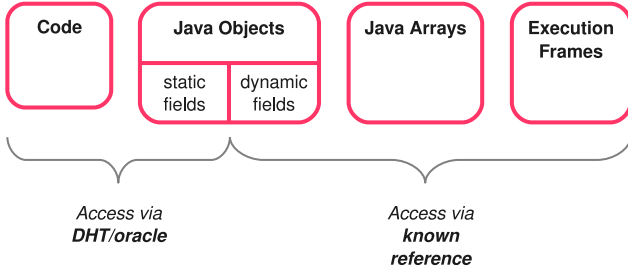


Fig. 3 Different types of GAOs.

the system that creates the SSI to adapt to changes in the network without the necessity for changes in the application code. In the AmbiComp application scenario multiple ACVMs collaborate to provide such a SSI.

All ACVM instances in a network of AICUs share (parts of) their local memory to form a network-wide distributed shared memory (DSM). We call entities in the DSM *global accessible objects (GAO)*. References to these GAOs are globally unique and independent of the GAOs' actual locations. The ACVMs can resolve these references to the respective locations in a fully decentralized manner [4].

Figure 3 shows the possible GAO types. They differ in two aspects: visibility of the referenced GAOs and way to obtain these references. Java objects and arrays are visible at Java level. Internal data, such as code and execution frames, is only visible to the ACVM. References to code or an object's static fields are obtained from an *oracle* (see below). References to object and array instances as well as execution frames are passed on between the ACVMs.

The oracle is used to find, access and organize the code and static field GAOs. It is implemented as a distributed hash table (DHT) [4]. The first ACVM instance that tries to initialize a class with static fields provides the oracle with the reference to a GAO that contains these static fields. Later on, ACVMs query the oracle for a reference to this GAO by providing the class' hash value and the application's domain (see below). This ensures that static fields of objects exist only once in each domain.

References to GAOs containing e.g. the dynamic fields of a Java object are obtained like any other Java reference: reading a field, passing a parameter, or using a method's return value.

The combination of both mechanisms e.g. reading a reference from a static field and following references in the reference graph, enables implicit communication between threads on remote ACVMs via shared memory.

As described so far, such a system would look like a single legacy VM. It executes multiple applications in

parallel without any isolation such as memory protection.

The lack of isolation between the different applications requires the programmer to take special care when accessing static fields because they exist only once in the DSM. JSR-121 [5] has introduced the concept of *isolates* to overcome this problem. Isolates completely separate the memory spaces of different applications executed by the same VM.

AmbiComp makes use of a similar concept. The so-called *domains* are used like isolates to separate different applications or groups of applications from each other. Applications in the same domain share static fields in the DSM. Applications in different domains can only use explicit communication to exchange data. They do not share their static fields in the DSM.

During the deployment phase applications are associated with their domain. Moreover, during this phase the same application code must be transferred to all ACVMs participating in the domain. This can be done by explicitly flashing the code into the AICUs. Another way to get hold of the code is to obtain a GAO reference to the code via the oracle by providing the hash value of the code and the application's domain. With this reference the ACVM can fetch the code from a remote location.

With the described approaches AmbiComp creates a SSI for the applications inside a domain. This frees the programmer from dealing with the explicit communication in those applications.

3.4 Example Application

Let us sketch a simple example of a distributed AmbiComp application. Suppose a scenario of an AICU *A* equipped with a temperature sensor and another AICU *B* equipped with a temperature sensor and an additional display device. Both AICUs are connected via a network connection such as ethernet, bluetooth or 802.15.4. Each of them executes a temperature measurement application. AICU *B* also executes an application that displays the temperature on the display. The measurement applications use the shared *TemperatureDataStore* object to store their measured values. The display application uses the *TemperatureDataStore* object to retrieve the data to be displayed.

As one can see in Fig. 4 all these applications exchange their data via the *TemperatureDataStore* object located in the distributed shared memory.

As we do not expect AICUs to start in a pre-determined order, it is unknown a priori which ACVM instance hosts the *TemperatureDataStore* object. This

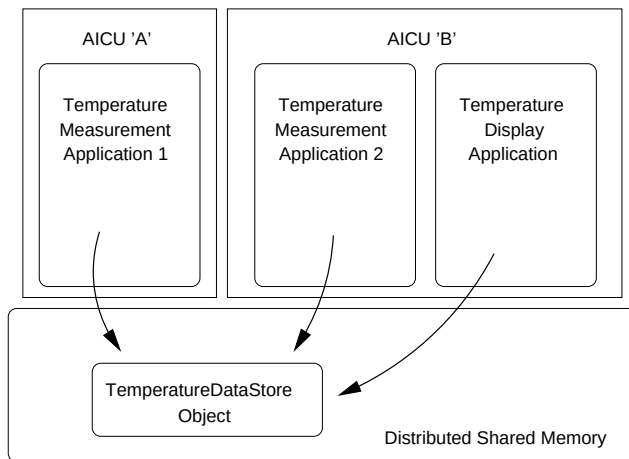


Fig. 4 Multiple applications using the shared distributed heap.

ambiguity is completely hidden from the developer. It is always possible to gain access to the object, for example, by querying the oracle for a static field containing the respective reference.

4 The Eclipse Plug-in

A main challenge when programming Ambient Intelligence Systems is to provide the developer with an easy-to-use development environment. One step towards this goal is the use of a high-level programming language. Therefore, the AmbiComp platform is based on Java.

Another important step is to provide the developers with a comfortable software development tool. Ideally it is based on an environment they are already experienced in. AmbiComp hence provides an Eclipse plug-in as shown in Fig. 5.

4.1 Eclipse and AmbiComp

The open source framework Eclipse [6] has proven to be an ideal base for all kinds of development tools because it is extensible through an easy-to-use plug-in mechanism. The Java Development Tool (JDT) is such a plug-in. It has evolved into being the quasi-standard for programming Java applications. The Plug-in Development Environment (PDE) plug-in enables easy development of plug-ins for Eclipse.

Other projects in the context of Ambient Intelligence also use Eclipse plug-ins as development tool. The d.tools-project for example uses new editors and views in Eclipse for rapid prototyping of HCI concepts [7]. Sun however uses its IDE Netbeans for programming SunSPOTs while providing an additional management tool for configuring, updating and simulating SunSPOTs and SunSPOT applications.

Eclipse plug-ins can, and most of them do, define so-called extension points. AmbiComp specific views, preferences, menus or wizards were created using these extension points of other Eclipse plug-ins and hence get integrated in the Eclipse framework very well. Moreover, Eclipse offers a framework to install and update plug-ins which is also used by the AmbiComp project to let any developer profit from recent releases.

4.2 The AmbiComp Plug-in

A major task of AmbiComp was to offer a single, easy-to-use development tool covering the whole toolchain. The plug-ins thus offer the following features:

- AmbiComp project creation using a wizard
- programming of AmbiComp applications in Java
- compiling, transcoding and deploying transcoded files to the AICUs
- ACVM updates
- some additional views (see Figure 6).

It is out of the scope of this paper to cover the complete plug-in architecture and features. Thus we describe only some major aspects here.

When programming for AmbiComp the developer can choose between two AmbiComp API flavors: a slightly modified implementation of the standard CLDC API creates a well-known base for Java applications; and a subset of the CLDC called miniAPI allows BLOBs with a significantly smaller memory footprint.

The miniAPI reduces the number of classes from 97 (CLDC) to just 27 and the lines of code from 21928 (CLDC) to 1141. It only implements the minimal set of classes necessary to compile and execute very basic applications. As the ACVM does not contain a pre-installed set of classes that typically constitute a Java runtime environment the BLOB contains the required subset of the chosen API. Using the miniAPI instead of the CLDC to create the BLOB further reduces its size. Making the miniAPI that small was possible by dropping features of the CLDC. This makes the miniAPI suitable for prove of concept applications only.

The AmbiComp project wizard includes the files of the chosen API automatically into the project. AmbiComp uses the Java Standard compiler, which is already integrated in Eclipse. As explained in section 3.1, all compiled class files of one project then have to be transcoded into the more efficient AmbiComp specific BLOB format. Software developers can initiate the transcoding process through a context sensitive menu on the AmbiComp project. They can then select and deploy the resulting BLOB on a physical or emulated AICU.

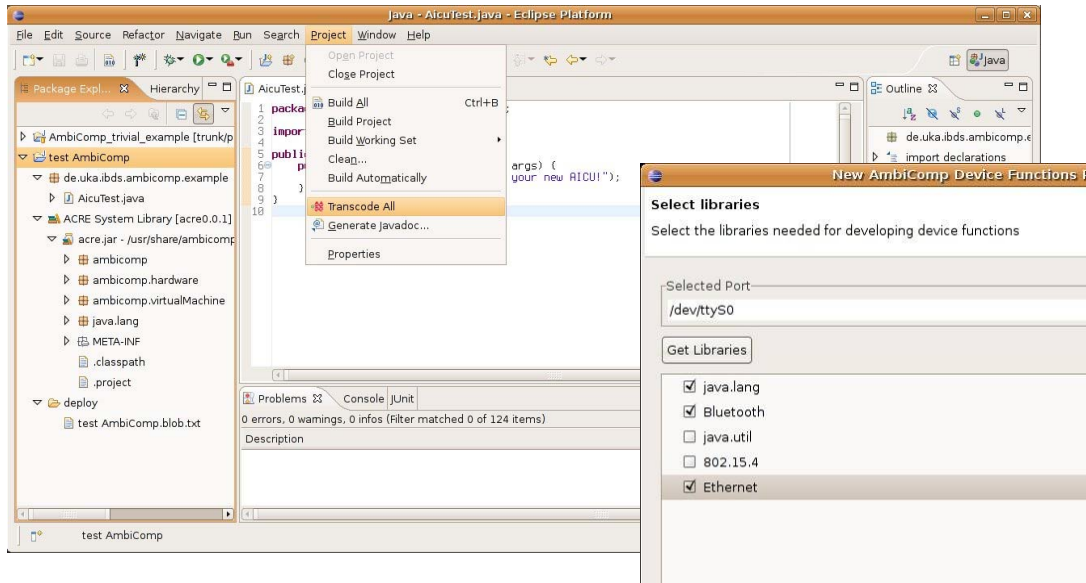


Fig. 5 AmbiComp Eclipse Plug-in Screenshot

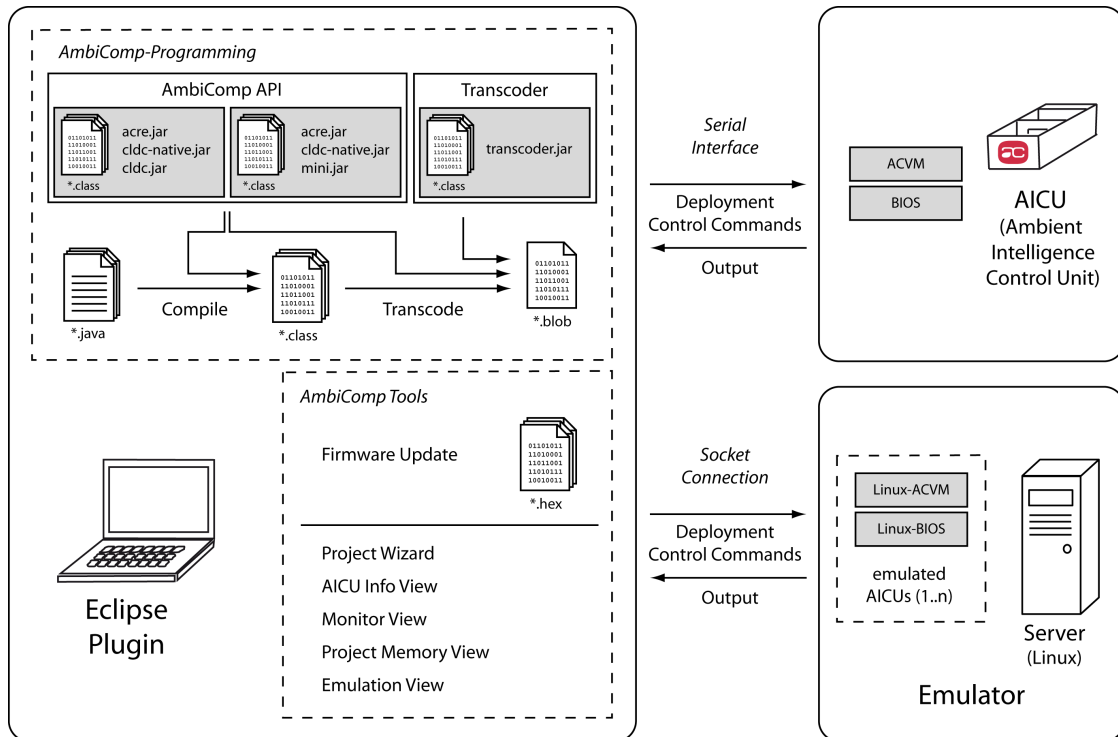


Fig. 6 Overview AmbiComp Eclipse Plug-in

4.3 The AICU Emulator

Another important component is the AmbiComp Emulator which runs on Linux servers (see Figure 6). The Eclipse plug-in communicates with these servers through a socket connection. The respective Emulation View lists all servers currently connected. It can be used

to create and control instances of Linux-ACVMs which behave just like physically connected AICUs. Additionally, real AICUs can be connected to the AmbiComp Emulator, too. They can also be controlled by the AmbiComp Eclipse plug-in. Real and emulated AICUs execute the same kind of BLOB files.

A couple of additional emulator related views support the developer. With the help of the Monitor View, a developer can monitor the entire communication between Eclipse, the physically connected AICUs, and the emulated AICUs (Linux-ACVMs). The AICU Info View shows some basic information about all the connected AICUs. The Project Memory View enables the developer to decide which project can be deployed on which AICU regarding memory requirements.

5 Conclusions and Future Work

This paper has given an overview over the current state of the AmbiComp project. It has described the hardware and software concepts that have been developed to support a wide range of application scenarios. It has also briefly introduced the currently available hardware modules and the Eclipse-based IDE.

The Ambient Intelligence Control Units (AICUs) consist of a combination of one or more sandwich modules with specific features. A hardware abstraction layer (“BIOS”) separates the AmbiComp Virtual Machine from the underlying hardware. On top of the BIOS a distributed AmbiComp Virtual Machine (ACVM) executes specifically transcoded Java applications. The ACVM is specially designed for 8-bit RISC micro-controllers. The ACVM also runs on Linux, so that a set of them can be used to emulate AICUs. The ACVM supports multi-threading and the execution of one or more different applications. Additionally it supports the distributed execution of an application beyond the SM and AICU boundaries by offering a Single System Illusion to the programmer. Currently, the AmbiComp platform does not support real time applications. The implementation of real time support is on-going work in our group.

For easy development of Ambient Intelligence applications, the AmbiComp platform supports application programming in the well known high-level programming language Java. An Eclipse plug-in offers all the functionalities that a developer needs in the AmbiComp context. It can, for example, be used to transcode and deploy the application on the AICUs. Furthermore, it helps to monitor the execution of applications both, in the AmbiComp Emulator and on the real hardware.

In future releases the Eclipse plug-in shall be able to request features from connected AICUs dynamically and integrate their representation in the various views. Later in the project, the Eclipse plug-in shall be extended towards a more system-wide perspective of AmbiComp. Additional views shall be created, e.g. to visualize an underlying AmbiComp network, to simplify

system-wide debugging, or to show system-wide memory conditions.

References

1. Ambicomp project. URL <http://ambicomp.org>
2. Specification of roboternetz bus plugs. URL <http://www.roboternetz.de/wissen/index.php/RN-Definitionen#RN-Busstecker>
3. Shaylor, N., Simon, D.N., Bush, W.R.: A java virtual machine architecture for very small devices. In: LCTES '03: Proceedings of the 2003 ACM SIGPLAN conference on Language, compiler, and tool for embedded systems, pp. 34–41. ACM Press, New York, NY, USA (2003). DOI <http://doi.acm.org/10.1145/780732.780738>
4. Saballus, B., Eickhold, J., Fuhrmann, T.: Global accessible objects (gaos) in the ambicomp distributed java virtual machine. In: Proceedings of the Second International Conference on Sensor Technologies and Applications (SENSORCOMM 2008). IEEE Computer Society, Cap Esterel, France (2008). URL <http://i30www.ira.uka.de/research/publications/p2p/>
5. Jsr-121 application isolation api specification (2006). URL <http://jcp.org/en/jsr/detail?id=121>
6. Eclipse platform. URL <http://www.eclipse.org>
7. Hartmann, B., Klemmer, S.R., Bernstein, M., Abdulla, L., Burr, B., Robinson-Mosher, A., Gee, J.: Reflective physical prototyping through integrated design, test, and analysis. In: UIST '06: Proceedings of the 19th annual ACM symposium on User interface software and technology, pp. 299–308. ACM (2006). DOI <http://doi.acm.org/10.1145/1166253.1166300>